

SecondWave

Un marché secondaire pour les positions de vault verrouillées,
avec un analyste de risque intégré

Track 2 — Lending Protocol · flavour Loaded
Devnet public · rippled 3.4.0-rc5 · xrp1.js@5.2.0-beta.1

équipe BFT-PARIS-26

Le capital est enfermé, et le risque bouge quand même

Un vault **closed-ended** verrouille les dépôts pendant toute la phase Investment.
VaultWithdraw → tecT00_SOON. Un déposant qui a besoin de son argent n'a **aucune issue** jusqu'à la RedemptionDate.

Subscription	Investment	Redemption
dépôt et retrait libres prêter est interdit	tout est bloqué on prête	retrait du capital + rendement plus de nouveau prêt

Et pendant ce verrouillage, **le risque se dégrade sans que le prix bouge**.
AssetsTotal ne change qu'au moment où le courtier déclare un impairment — et **rien ne l'y oblige**.

Un prêt défaillable depuis des heures laisse la NAV parfaitement intacte.

Les parts sont un jeton : la sortie existe, elle n'est pas outillée

Les parts d'un vault sont un **MPT** (XLS-33). Elles se transfèrent. On outille la cession en trois briques :

- **Un carnet d'offres** — n'affiche à un acheteur que ce qu'il peut *réellement* recevoir, et dit *pourquoi* quand une offre lui est fermée.
- **Un règlement atomique** — parts contre prix, ensemble ou pas du tout, avec preflight, reconstruction du résultat et réconciliation des soldes.
- **Un analyste** — le vendeur brade toujours. Toute la question est *pourquoi*.

Décote de liquidité

le fonds va bien, le vendeur a besoin de cash
une opportunité

Décote de détresse

le fonds va mal, ça ne se voit pas encore
un piège

prix affiché : identique

Le flux on-chain — six normes XLS combinées

XLS-70+80 CredentialCreate → CredentialAccept → PermissionedDomainSet

XLS-65 VaultCreate (*privé, closed-ended*) → VaultDeposit

XLS-66 LoanBrokerSet + CoverDeposit → LoanSet → LoanPay / LoanManage

⇓ **bascule en Investment — les retraits sont bloqués**

XLS-33+56 le Batch tfAllOrNothing en trois jambes :

MPTokenAuthorize → Payment des parts → Payment du prix

18 types de transaction employés, chacun avec un lien on-chain vérifié dans le README.

Le gate du domaine s'applique partout : un non-membre reçoit tecNO_AUTH au dépôt, au transfert direct, et même au dénouement d'un escrow.

Démonstration

De l'offre au règlement vérifié

`npm run cli vaults` · `npm run cli book` · `npm run cli buy`

Friction 1 — un Batch ne dit pas s'il a produit un effet

Réponse complète du serveur sur un Batch qui n'a *rien* fait :

```
accepted: true | applied: true | engine_result: tesSUCCESS  
engine_result_message: "The transaction was applied."  
meta.AffectedNodes: [ 1 noeud ] ← la ponction de frais
```

Sur **huit** Batch rejoués — cinq ayant déplacé des parts, trois n'ayant rien fait — **une seule combinaison apparaît**. Succès et non-effet sont indiscernables. Et simulate répond notImpl sur un Batch : **ni pré-jouer, ni constater**.

Ce qu'on demande — le BatchExecutions que XLS-56 spécifie déjà. À défaut, **l'index de la jambe en échec et son code**. Un entier et un code.

L'amendement est encore en BatchV1_1 sur une *release candidate* : c'est maintenant que ça coûte le moins cher.

Friction 2 — trois drapeaux sur quatre livrent sans encaisser

Le même échange, jambe du prix volontairement impayable, rejoué sous chaque drapeau :

Drapeau	Annoncé	Parts déplacées	Prix payé
tfAllOrNothing	tesSUCCESS	0	0
tfOnlyOne · tfUntilFailure	tesSUCCESS	300 000	0
tfIndependent			

L'oubli est impossible — Flags: 0 est refusé. Le risque, c'est **le mauvais choix**, fait en connaissance de cause. Et combiné à la friction 1, il est **indétectable** : même code, même métadonnée, et l'actif est parti.

Ce qu'on demande — une mise en garde dans XLS-56 pour les échanges de valeur, et un helper de swap dans les SDK qui *impose* tfAllOrNothing.

Friction 3 — aucune primitive de lecture côté XLS-66

`vault_info` existe. **Ni** `loan_info`, **ni** `loan_broker_info`, **ni** la liste des détenteurs de parts.

```
vault_info(vaultId)
  → account_objects(pseudo-compte du vault, type: 'loan_broker')
  → account_objects(pseudo-compte du courtier, type: 'loan')
```

Et pour savoir qui détient les parts : **rejouer tout l'historique** du pseudo-compte et reconstruire les soldes depuis la métadonnée. C'est la partie du code qui nous a demandé le plus de travail — et elle ne reconstitue que ce que le serveur sait déjà.

Ce qu'on demande — `loan_info` et `loan_broker_info` sur le modèle de `vault_info`, et une commande qui liste les détenteurs d'une émission MPT.

Ce qu'on a mesuré

Cas exécutés sur le Devnet public	~ 420 sur une trentaine de vaults
Coût d'un échange complet	200 drops — 0,0152 % du prix
Décote de marché observée	12 à 14 % selon le vault
→ le rail coûte 800 fois moins cher que la décote	
Charger l'état complet d'un vault	4 appels RPC, ~700 ms
Bug bloquant trouvé	LoanSet inutilisable — corrigé en 5.2.0-beta.1

Et ce qui nous a rassurés

- **Personne ne peut enfermer un déposant** : les dates ne sont pas modifiables après création.
- **Le pseudo-compte est étanche** — paiement, escrow, chèque : `tecNO_PERMISSION`.
- **Aucune fuite d'arrondi** sur 145 aller-retours en XRP et en IOU.
- `tfAllOrNothing` **tient** dans tous les cas d'échec testés.

Le carnet d'ordres natif connaîtra-t-il les **Permissioned Domains** ?

OfferCreate avec un MPT répond `temDISABLED` — pas `temMALFORMED`. Le code existe, l'amendement dort. C'est pour ça que notre carnet est hors chaîne.

Mais un carnet d'ordres est **sans permission** : il apparie deux ordres sans savoir qui les a posés. Nos parts sont gâtées. Un DEX qui accepterait les MPT sans connaître les domaines apparierait des ordres dont le règlement serait ensuite refusé.

Si le DEX connaît les domaines	notre carnet devient inutile
S'il ne les connaît pas	un marché P2P reste la seule voie pour un vault privé

Dans les deux cas, l'analyste reste : un carnet affiche un prix, il ne dit pas si c'est une aubaine ou un piège.